

ANSIBLE

Architecture, Environment Setup
& Static Inventory Management

Professional Reference Documentation

Version 1.0

Table of Contents

- 1. Introduction
- 2. Ansible Architecture
 - 2.1 Control Node
 - 2.2 Managed Nodes
 - 2.3 The Agentless Push Model
 - 2.4 Connection Methods: SSH and WinRM
- 3. Environment Setup
 - 3.1 Prerequisites
 - 3.2 Installing Ansible
 - 3.3 SSH Key-Based Authentication
 - 3.4 The ansible.cfg File
 - 3.5 Configuration Precedence
 - 3.6 Essential ansible.cfg Parameters
- 4. Static Inventory Management
 - 4.1 Inventory Overview
 - 4.2 INI Format
 - 4.3 YAML Format
 - 4.4 Host Groups
 - 4.5 Parent-Child Group Relationships
 - 4.6 Group and Host Variables
 - 4.7 Verifying the Inventory
- 5. Best Practices
- 6. Quick Reference

1. Introduction

Ansible is an open-source IT automation engine used for configuration management, application deployment, orchestration, and infrastructure provisioning. Its defining characteristic is simplicity: it requires no agent software on the systems it manages and communicates over standard, encrypted transports that already exist in the environment.

This document covers two foundational areas of the platform:

- **Ansible architecture and environment setup** — the control node, managed nodes, the agentless push model, SSH and WinRM transports, and the `ansible.cfg` configuration file.
- **Static inventory management** — structuring host files in INI and YAML formats, defining host groups, and building parent-child group hierarchies.

The material assumes a working knowledge of the Linux command line and basic TCP/IP networking. All examples are compatible with Ansible Core 2.14 and later.

2. Ansible Architecture

2.1 Control Node

The control node is the machine on which Ansible is installed and from which all automation is executed. It is the origin of every playbook run and holds the complete automation toolchain.

- **Ansible installation** — command-line tools including `ansible`, `ansible-playbook`, `ansible-inventory`, and `ansible-galaxy`.
- **Automation content** — playbooks, roles, collections, and modules.
- **Inventory** — the list of managed nodes, in static file or dynamic source form.
- **Configuration** — the `ansible.cfg` file that governs runtime behaviour.

Any Unix-like operating system with Python 3.8 or later can serve as a control node: Linux distributions, macOS, and Windows Subsystem for Linux (WSL). Windows itself is not supported as a control node. A single control node can manage thousands of managed nodes; multiple control nodes may be deployed for scale or separation of duties.

2.2 Managed Nodes

Managed nodes are the target systems that Ansible configures. They require only the following:

- Network reachability from the control node.
- A valid user account with appropriate privileges.
- A Python 3 interpreter (Linux / Unix) or PowerShell 3.0 or later (Windows).
- No Ansible agent, daemon, or proprietary software installed.

For each task, Ansible transfers a small, self-contained module to the managed node, executes it, collects the JSON result, and removes the module. This transient execution model is central to the architecture and eliminates the maintenance burden of a persistent agent.

2.3 The Agentless Push Model

Ansible operates on a **push** model: the control node initiates every connection to the managed nodes. Managed nodes never poll a central server and never hold credentials for the control node.

Execution flow for a single task:

1. Control node reads the inventory and the playbook.
2. Control node opens a connection to the managed node (SSH or WinRM).
3. Control node transfers the required module (a small Python or PowerShell file).
4. The module executes locally on the managed node and returns JSON.
5. The temporary module file is removed from the managed node.
6. The control node records the result (ok / changed / failed / skipped).

Because modules are written to be **idempotent**, they describe a desired state rather than a sequence of commands. Running the same playbook repeatedly produces no unintended changes once the target has converged.

2.4 Connection Methods: SSH and WinRM

Ansible communicates with managed nodes over two principal transports. The choice is determined by the operating system of the target.

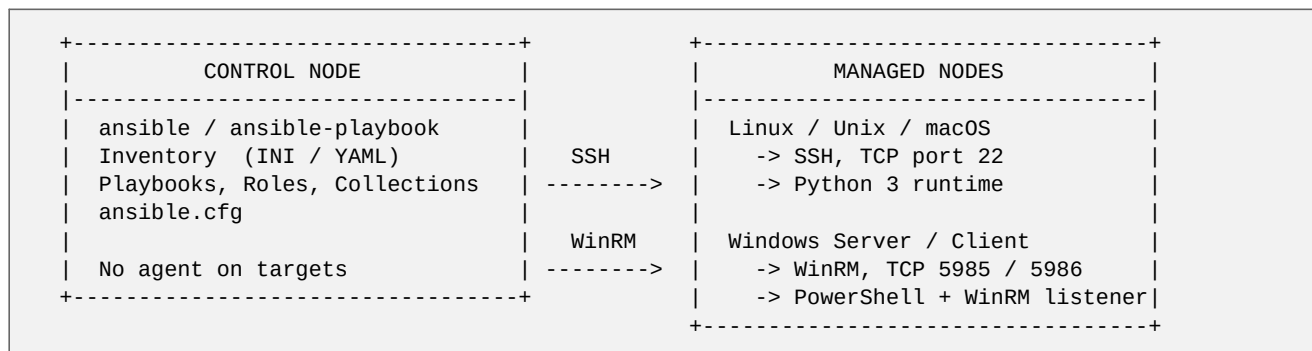


Figure 1 — Ansible agentless push architecture.

SSH (Linux, Unix, macOS, network devices)

- The default and preferred transport for POSIX systems.
- Uses the system OpenSSH client; key-based authentication is strongly recommended over passwords.
- Performance is improved by ControlPersist (connection multiplexing) and pipelining (fewer round trips).
- Common host variables: `ansible_user`, `ansible_port`, `ansible_ssh_private_key_file`.

WinRM (Windows)

- Windows Remote Management, a SOAP-based protocol over HTTP or HTTPS.
- Requires PowerShell and a configured WinRM listener on the target.
- Requires the `pywinrm` Python library on the control node.
- Common host variables: `ansible_connection=winrm`, `ansible_winrm_transport`, `ansible_port=5986` for HTTPS.

- PSRP (`ansible_connection=psrp`) is an alternative connection plugin that runs over WinRM and offers improved performance and error handling.

Local connection

The `local` connection plugin executes tasks on the control node itself without opening a network connection. It is used for tasks such as API calls, cloud provisioning, and inventory manipulation.

Aspect	SSH	WinRM
Target OS	Linux, Unix, macOS, network devices	Windows Server / Client
Default port	22 (TCP)	5985 HTTP / 5986 HTTPS
Authentication	SSH key pair (preferred), password	Kerberos, NTLM, CredSSP, certificate, basic
Control-node dependency	OpenSSH client (standard)	pywinrm Python library
Target dependency	SSH daemon, Python 3	WinRM listener, PowerShell 3.0+
Connection variable	<code>ansible_connection=ssh</code>	<code>ansible_connection=winrm</code>
Key tuning	<code>ControlPersist</code> , <code>pipelining</code> , <code>ssh_args</code>	<code>ansible_winrm_transport</code> , <code>ansible_winrm_server_cert_validation</code>

Table 1 — Comparison of SSH and WinRM transports.

3. Environment Setup

3.1 Prerequisites

- Control node running Linux, macOS, or WSL with Python 3.8 or later.
- An OpenSSH client available on the control node.
- Network reachability to all managed nodes on the chosen transport port.
- A user account on each managed node with the privileges required for the intended tasks (often escalated with become).
- For Windows targets: pywinrm installed on the control node.

3.2 Installing Ansible

Ansible is distributed as a Python package. Installation on the control node is performed with the platform package manager or with pip.

```
# Debian / Ubuntu
sudo apt update
sudo apt install -y ansible

# RHEL / Rocky Linux / AlmaLinux
sudo dnf install -y ansible-core

# macOS (Homebrew)
brew install ansible

# pip (any platform -- recommended for pinning versions)
python3 -m pip install --user ansible

# Verify the installation
ansible --version
```

The `ansible-core` package contains the engine and a minimal set of built-in modules. The full `ansible` package additionally bundles a large collection of community modules and roles.

3.3 SSH Key-Based Authentication

Key-based authentication removes the need for interactive passwords and is the standard practice for Ansible control nodes.

```
# 1. Generate a dedicated key pair on the control node (no passphrase for automation)
ssh-keygen -t ed25519 -f ~/.ssh/ansible_ed25519 -C "ansible-control"

# 2. Distribute the public key to each managed node
ssh-copy-id -i ~/.ssh/ansible_ed25519.pub ansible@web01.example.com

# 3. Verify passwordless login
ssh -i ~/.ssh/ansible_ed25519 ansible@web01.example.com 'hostname'
```

A passphrase-protected key may be used together with `ssh-agent`. For unattended automation, either a passphrase-less key or an agent-backed key is required.

3.4 The `ansible.cfg` File

The `ansible.cfg` file controls the default behaviour of every Ansible command. It uses the INI format, organised into sections such as `[defaults]`, `[privilege_escalation]`, `[ssh_connection]`, and `[inventory]`.

3.5 Configuration Precedence

When multiple configuration files are present, Ansible uses the first one it finds, in the following order of precedence (highest first):

Order	Location	Notes
1	<code>\$ANSIBLE_CONFIG</code>	Environment variable pointing to an explicit file.
2	<code>./ansible.cfg</code>	Project directory. Preferred for version-controlled projects.
3	<code>~/.ansible.cfg</code>	Per-user defaults for interactive use.
4	<code>/etc/ansible/ansible.cfg</code>	System-wide default, typically installed by the package.

Table 2 — Configuration file precedence.

Best practice: keep a project-local `ansible.cfg` in the root of the automation repository so that every engineer and CI job uses identical settings.

3.6 A Representative `ansible.cfg`

```

[defaults]
inventory          = ./inventory/hosts.yml
remote_user        = ansible
private_key_file    = ~/.ssh/ansible_ed25519
host_key_checking   = False
retry_files_enabled = False
forks              = 20
timeout            = 30
gathering           = smart
fact_caching        = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400
stdout_callback     = yaml
roles_path          = ./roles
collections_path    = ./collections
log_path           = ./ansible.log

[privilege_escalation]
become              = True
become_method       = sudo
become_user         = root
become_ask_pass     = False

[ssh_connection]
pipelining          = True
ssh_args            = -o ControlMaster=auto -o ControlPersist=60s
control_path        = /tmp/ansible-ssh-%%h-%%p-%%r

[persistent_connection]
connect_timeout     = 30
command_timeout     = 30

[inventory]
enable_plugins      = host_list, script, auto, yaml, ini, toml

```

Security note: disabling host key checking (`host_key_checking = False`) is acceptable only in controlled lab environments. In production, manage `known_hosts` explicitly.

Section	Parameter	Purpose	Typical value
defaults	inventory	Default inventory source.	./inventory/hosts.yml
defaults	remote_user	Default SSH / WinRM user.	ansible
defaults	private_key_file	Default SSH private key.	~/.ssh/id_ed25519
defaults	forks	Parallel processes.	20
defaults	gathering	Fact-gathering policy.	smart
defaults	stdout_callback	Console output format.	yaml
defaults	log_path	Execution log location.	./ansible.log
privilege_escalation	become	Enable privilege escalation.	True
privilege_escalation	become_method	Escalation mechanism.	sudo
ssh_connection	pipelining	Reduce SSH round trips.	True
ssh_connection	ssh_args	Connection multiplexing.	ControlPersist=60s

Section	Parameter	Purpose	Typical value
inventory	enable_plugins	Permitted inventory formats.	yaml, ini, ...

Table 3 — Essential ansible.cfg parameters.

4. Static Inventory Management

4.1 Inventory Overview

The inventory is the authoritative list of managed nodes that Ansible can address. It maps logical names to connection details and organises hosts into groups that playbooks can target directly.

- **Static inventory** — a text file maintained by hand, in INI or YAML format. Suitable for stable environments and for learning.
- **Dynamic inventory** — generated at runtime by an inventory plugin or script that queries a source of truth such as a cloud provider or CMDB. (Outside the scope of this document.)

The default system location is `/etc/ansible/hosts`, but a project-local inventory referenced from `ansible.cfg` is strongly preferred.

4.2 INI Format

The INI format is compact and widely used. Hosts are listed on their own lines beneath a bracketed group header. Host-specific variables are supplied as `key=value` pairs on the host line or in a `[group:vars]` section.

```
# inventory/hosts.ini

# --- Ungrouped hosts -----
web01.example.com
192.0.2.10

# --- Functional groups -----
[webservers]
web01.example.com
web02.example.com
web03.example.com ansible_host=192.0.2.53

[databases]
db01.example.com
db02.example.com

[loadbalancers]
lb01.example.com

# --- Environment group built from other groups -----
[production:children]
webservers
databases
loadbalancers

[production:vars]
ansible_user=deploy
ansible_port=22

# --- Sequential and alphabetic ranges -----
[web_cluster]
web[01:10].example.com

[db_cluster]
db-[a:c].example.com
```

Ranges such as `web[01:10].example.com` expand to ten hosts without listing each one. Alphabetic ranges are also supported.

4.3 YAML Format

The YAML format expresses the same structure as nested mappings and is preferred for complex inventories because its hierarchy is explicit and it validates cleanly with standard YAML tooling.

```
# inventory/hosts.yml
all:
  hosts:
    standalone.example.com:
  children:

    webservers:
      hosts:
        web01.example.com:
        web02.example.com:
        web03.example.com:
        ansible_host: 192.0.2.53

    databases:
      hosts:
        db01.example.com:
        db02.example.com:

    loadbalancers:
      hosts:
        lb01.example.com:

    production:
      children:
        webservers:
        databases:
        loadbalancers:
      vars:
        ansible_user: deploy
        ansible_port: 22
```

The two formats are functionally equivalent. Choose INI for brevity in small environments and YAML for clarity and maintainability at scale.

4.4 Host Groups

- Groups organise hosts by function (webservers, databases), location (atlanta, raleigh), or environment (production, staging).
- `all` is an implicit group that contains every host in the inventory.
- `ungrouped` is an implicit group for hosts that have not been assigned to any other group.
- A host may belong to any number of groups simultaneously.
- Group names should use letters, digits, and underscores. Hyphens are converted to underscores internally and should be avoided.

4.5 Parent-Child Group Relationships

A parent group is a group of groups. It is defined with the `:children` suffix in INI or the `children:` key in YAML. Hosts are never directly listed in a parent group; instead, the parent references child groups.

Key properties:

- **Variable inheritance** — variables defined on a parent group are inherited by every host in every descendant group.

- **Targeting** — a playbook that targets a parent group automatically addresses all hosts in all of its children.
- **Single root** — the `all` group is the ultimate parent of every group in the inventory.
- **Layering** — hierarchies are commonly used to layer environment, region, and role.
- **No host duplication** — a host appears once and is reached through as many groups as required.

Example hierarchy in INI format:

```
[atlanta]
host1.example.com
host2.example.com

[raleigh]
host3.example.com
host4.example.com

[southeast:children]
atlanta
raleigh

[southeast:vars]
ntp_server=ntp.southeast.example.com

[production:children]
southeast

[production:vars]
ansible_user=deploy
env=production
```

The equivalent hierarchy in YAML format:

```
all:
  children:
    production:
      children:
        southeast:
          children:
            atlanta:
              hosts:
                host1.example.com:
                host2.example.com:
            raleigh:
              hosts:
                host3.example.com:
                host4.example.com:
          vars:
            ntp_server: ntp.southeast.example.com
        vars:
            ansible_user: deploy
            env: production
```

A host in `atlanta` therefore inherits `ntp_server` from `southeast` and `ansible_user` and `env` from `production`, without any of those variables being repeated.

4.6 Group and Host Variables

Variables may be defined inline in the inventory, but for anything beyond the simplest case they should be placed in dedicated `group_vars/` and `host_vars/` directories adjacent to the inventory file.

```
inventory/
|-- hosts.yml
|-- group_vars/
|   |-- all.yml           # applies to every host
|   |-- webservers.yml    # applies to the webservers group
|   |-- production.yml    # applies to the production group
|-- host_vars/
|   |-- web01.example.com.yml # applies to one host only
```

Variable precedence, from lowest to highest, is: `group_vars/all`, then parent group variables, then child group variables, then `host_vars`. More specific definitions always win.

4.7 Verifying the Inventory

Ansible provides the `ansible-inventory` command for inspecting and validating inventory structure before any playbook is run.

```
# List every host and variable as JSON
ansible-inventory --list -i inventory/hosts.yml

# Render the group hierarchy as a tree
ansible-inventory --graph -i inventory/hosts.yml

# Show the resolved variables for a single host
ansible-inventory --host web01.example.com -i inventory/hosts.yml

# Confirm connectivity to all hosts
ansible all -i inventory/hosts.yml -m ping

# Confirm connectivity to one group
ansible webservers -i inventory/hosts.yml -m ping
```

The `--graph` output is the quickest way to confirm that parent-child relationships have been constructed as intended.

5. Best Practices

- **Version everything.** Keep the inventory, `ansible.cfg`, playbooks, and roles in a single Git repository so that the environment is fully reproducible.
- **Use a project-local `ansible.cfg`.** This guarantees identical behaviour across engineers and CI runners.
- **Prefer `group_vars` and `host_vars`.** Keep the inventory file itself focused on structure, not on variables.
- **Model environments with parent groups.** Build a hierarchy such as `all -> production -> region -> role`, and let variables cascade.
- **Choose names carefully.** Use letters, digits, and underscores in group names. Avoid hyphens, spaces, and ambiguous abbreviations.
- **Never store plaintext secrets.** Use `ansible-vault` for sensitive variables and SSH keys for authentication.
- **Validate before execution.** Run `ansible-inventory --graph` and `ansible all -m ping` after any inventory change.
- **Enable pipelining.** It substantially reduces the number of SSH round trips per task.
- **Tune forks deliberately.** Higher concurrency shortens runtime but increases load on the control node and on network devices.
- **Restrict host key checking in production.** Disable it only in isolated test environments.

6. Quick Reference

Command	Purpose
<code>ansible --version</code>	Display the installed version and active configuration file.
<code>ansible all -m ping</code>	Verify connectivity and authentication to every host.
<code>ansible all -m setup</code>	Collect and display facts from all hosts.
<code>ansible-inventory --list</code>	Dump the inventory as JSON, including resolved variables.
<code>ansible-inventory --graph</code>	Render the group hierarchy as a tree.
<code>ansible-inventory --host</code>	Show every variable resolved for one host.
<code>ansible-playbook site.yml --syntax-check</code>	Validate playbook syntax without executing it.
<code>ansible-playbook site.yml --check</code>	Dry run: report changes without applying them.
<code>ansible-playbook site.yml --limit webservers</code>	Restrict a run to a group or host pattern.
<code>ansible-vault encrypt group_vars/all.yml</code>	Encrypt a variables file with a vault password.
<code>ansible-config dump --only-changed</code>	Show configuration values that differ from the defaults.

Table 4 — Frequently used commands.

This document describes Ansible Core behaviour as of version 2.14 and later. Always consult the official documentation for the version in use.